SafeSort-RL

Language-Conditioned Hazardous-Waste Sorting on AMD Radeon GPUs and ROCm

Track 3: Physical AI Challenge - Robotics Simulation and Application Design

Junie Mariam Varghese | Solo Submission | August 2026

Abstract

SafeSort-RL is a language-conditioned robotic sorting system developed for the Track 3 Physical AI Challenge. The application controls a simulated Franka Panda arm in Genesis World and assigns four representative waste objects - a battery, chemical bottle, plastic bottle, and syringe - to either a hazard or recycle bin. A single Proximal Policy Optimization (PPO) policy receives a compact scene-state observation together with an encoded task instruction and outputs continuous Cartesian motion commands. Genesis inverse kinematics converts these commands into robot joint targets, while a deterministic safety layer provides reliable attachment and controlled release during manipulation. The simulator is executed on an AMD Radeon GPU through ROCm/HIP, accelerating rigid-body dynamics, inverse kinematics, scene stepping, and rendering. The final policy was evaluated over 100 balanced, fixed-seed episodes and achieved 81% overall success, compared with 0% for a random-action baseline. The strongest task reached 100% success, while battery sorting remained the principal failure mode at 44%. The project contributes an auditable evaluation protocol, a same-scene instruction-swap grounding probe, a reproducible AMD-native workflow, and a polished physical showcase with synchronized status visualization.

Keywords - *physical AI, robotic manipulation, reinforcement learning, PPO, Genesis World, Franka Panda, AMD Radeon, ROCm, hazardous-waste sorting*

1. Introduction and Target Application

Waste sorting is an appropriate physical-AI problem because it combines language-conditioned decision making, spatial reasoning, motion planning, and safe interaction with objects. In real facilities, batteries, chemical containers, syringes, and recyclable plastics should not be handled identically. A robotic system must identify the requested item, approach it without disturbing the remaining scene, transport it safely, and release it at the correct destination.

SafeSort-RL addresses this problem in simulation. The user provides a short natural-language command, such as “Move the battery to the hazard bin.” The command is normalized to one of four supported task intents. A single trained policy then moves a simulated Franka Panda arm toward the named object and places it in the required bin. The central research question is whether one policy can produce distinct behavior from the instruction while operating reliably on an AMD Radeon and ROCm simulation stack.

Table 1. Task definitions and destinations

Object class	Canonical instruction	Destination
Battery	Move the battery to the hazard bin.	Hazard bin
Chemical bottle	Move the chemical bottle to the hazard bin.	Hazard bin
Plastic bottle	Move the plastic bottle to the recycle bin.	Recycle bin
Syringe	Move the syringe to the hazard bin.	Hazard bin

Application boundary

The system is a hybrid language-conditioned reinforcement-learning application. The PPO policy learns continuous reaching and transport behavior. Instruction normalization, object attachment, and release are deterministic components. The reported evidence is simulation-only and does not claim end-to-end language understanding or real-robot deployment.

2. Overall System Architecture and Solution Design

The design separates task interpretation, learned motion, robot control, and grasp safety into explicit stages. This separation makes the system easier to evaluate and prevents success from being attributed to an opaque end-to-end controller. The complete processing path is shown in Figure 1.

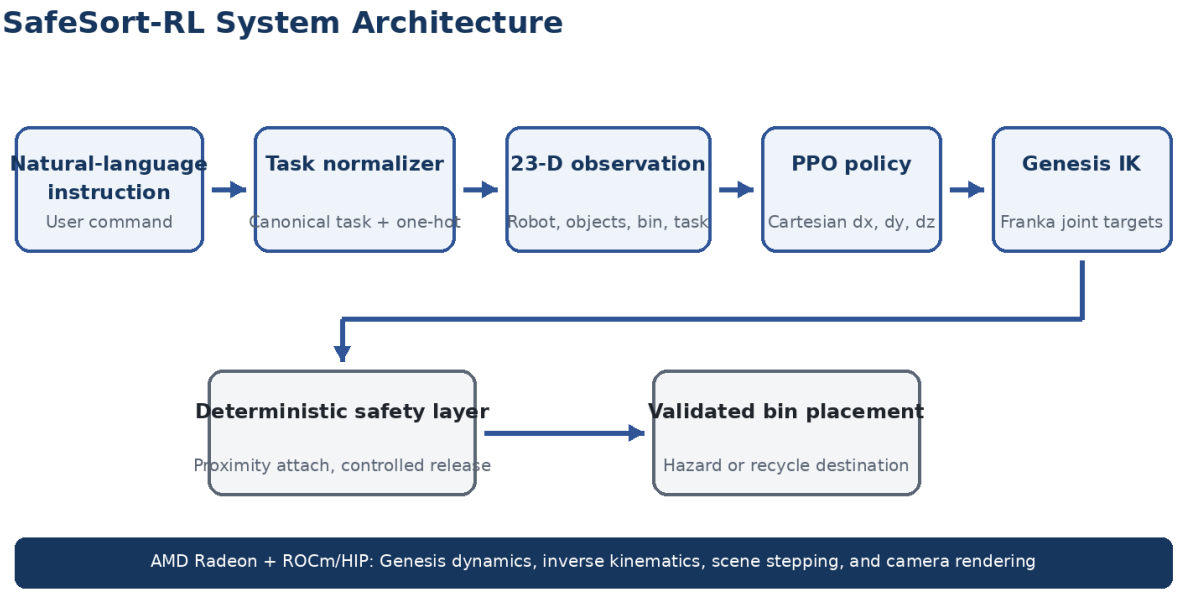


Figure 1. Instruction-conditioned control pipeline and AMD-accelerated simulation boundary.

2.1 Observation and action spaces

Each control step uses a 23-dimensional observation. The vector contains the gripper position (3 values), the positions of four objects (12 values), the target-bin position (3 values), a four-element one-hot task encoding, and a one-element holding flag. The policy outputs a three-dimensional Cartesian displacement, (dx, dy, dz), rather than direct joint torques. Genesis inverse kinematics transforms that displacement into Franka Panda joint targets.

2.2 Manipulation and safety layer

The learned policy determines how the end effector approaches and transports the selected item. A deterministic proximity rule attaches the object only after the end effector enters an object-specific grasp region. Release is performed at a validated point inside the destination bin, followed by a controlled retreat. This safety layer improves repeatability and makes failure analysis clearer, while preserving the learned role of the PPO controller in reaching and transport.

2.3 Main software components

Table 2. System components

Layer	Implementation	Role
Simulation	Genesis World 1.3.1	Rigid-body physics, scene stepping, IK, rendering
Robot	Franka Emika Panda (MJCF)	Nine-DoF arm and gripper model
Policy	PPO MlpPolicy, Stable-Baselines3	Instruction-conditioned Cartesian motion
Task interface	Rule-based command normalizer	Maps user wording to four canonical tasks

Layer	Implementation	Role
Safety layer	Proximity weld and controlled release	Reliable grasp attachment and bin placement
Visualization	Genesis camera + OpenCV panel	Synchronized demonstration and status display

3. Training Data and Evaluation Methodology

SafeSort-RL does not rely on an external image, language, or demonstration dataset. Training experience is generated on-policy inside the simulator. At the beginning of each episode, the environment selects a task instruction and places the objects with controlled random variation. This produces a stream of state-action-reward transitions that reflects the current PPO policy and exposes it to varied layouts.

3.1 Reward design and curriculum

The shaped reward provides incremental feedback for progress toward the selected object, a grasp bonus after successful attachment, progress toward the correct bin while the object is held, and a larger terminal reward for a correct placement. A small per-step penalty encourages efficient behavior. Training was organized as a three-stage curriculum: a stable baseline, a targeted stage emphasizing weaker tasks, and a final balancing stage that reduced task imbalance while preserving the strongest behaviors.

3.2 Evaluation protocol

Final evaluation uses 100 balanced, fixed-seed episodes: 25 episodes for each of the four tasks. Evaluation seeds are separated from the training starts used during development. This protocol prevents a high-frequency task from dominating the aggregate result and makes the per-task weaknesses visible. A random-action policy is evaluated as a baseline under the same task structure.

- Primary metric: task success rate after correct placement in the assigned bin.
- Secondary metrics: episode reward, episode length, and final distance to the target bin.
- Grounding check: a same-scene instruction-swap probe changes only the command and observes whether the selected target changes.
- Showcase validation: each object is physically attached, transported, released, and checked for final containment before success is displayed.

4. AMD Radeon GPU and ROCm Utilization

The primary computational workload is the Genesis simulation. The environment is initialized with the AMD GPU backend, and Genesis reports its active simulation device through the ROCm/HIP stack. Rigid-body dynamics, collision handling, Franka inverse kinematics, scene stepping, and camera rendering are therefore executed on the Radeon GPU. This acceleration is used during training because every PPO rollout requires repeated simulator steps, and it is also used during evaluation and video generation.

The final notebook verifies that PyTorch is a ROCm build by checking torch.version.hip and confirms that an AMD GPU is visible through the torch.cuda-compatible API. In this software stack, the device label cuda:0 is an API compatibility name for the active HIP/ROCm device rather than evidence of an NVIDIA GPU. The compact PPO policy is loaded on CPU for final evaluation, while the expensive simulation and rendering path remains on Radeon. This division is appropriate for a small multilayer perceptron whose inference cost is minor compared with physics and rendering.

Table 3. Verified execution environment and GPU role

Item	Verified value	Use in SafeSort-RL
GPU	AMD Radeon Graphics	Genesis physics, IK, stepping, rendering
PyTorch	2.8.0 + ROCm 6.4	ROCm availability verification and model runtime

Item	Verified value	Use in SafeSort-RL
HIP	6.4.43482-0f2d60242	AMD GPU compute interface
Genesis	1.3.1	Simulation and robot control
Stable-Baselines3	2.9.0	PPO training, checkpointing, evaluation
Policy device	CPU in final evaluation	Low-cost MLP inference
Genesis device	cuda:0 compatibility label	Active ROCm/HIP simulation device

Reproducibility evidence

The notebook exports the Python, PyTorch, HIP, GPU, Genesis, and Stable-Baselines3 versions to machine-readable and text files. It can also capture a live rocm-smi snapshot. These artifacts document the exact AMD environment used to generate the reported evidence.

5. Innovations and Key Technical Contributions

- **Single policy for four task intents.** One PPO policy uses the instruction encoding to select and transport different objects instead of relying on one separately trained controller per object.
- **Same-scene instruction-swap probe.** The scene is held fixed while only the command changes. A change in the approached object provides direct evidence that behavior is conditioned on the instruction.
- **Auditable curriculum and evaluation.** The workflow records the progression from weaker chemical-bottle and syringe performance to a balanced final checkpoint, and reports per-task results rather than only an aggregate score.
- **AMD-native simulation workflow.** Genesis physics, IK, stepping, and rendering run on Radeon through ROCm/HIP during rollout generation, evaluation, and video production.
- **Physical showcase with honest claim boundaries.** The final video visibly closes the gripper, attaches the dynamic object, transports it, releases it inside the bin, verifies containment, and displays success only after the arm clears the workspace.
- **Procedural object visualization.** Multi-part visual skins make the battery, bottles, and syringe recognizable without changing the validated collision proxies that support stable manipulation.

6. Experimental Results and Analysis

The final checkpoint was evaluated over 100 balanced episodes. Table 4 and Figure 2 summarize the results. Overall success reached 81%, while the random-action baseline achieved 0%. The gap between the trained policy and the baseline confirms that the final behavior is not explained by favorable initial placement or unstructured motion.

Table 4. Balanced task success rates

Task	Episodes	Success rate
Battery -> hazard bin	25	44%
Chemical bottle -> hazard bin	25	92%
Plastic bottle -> recycle bin	25	88%
Syringe -> hazard bin	25	100%
Overall	**100**	**81%**
Random-action baseline	Balanced	0%

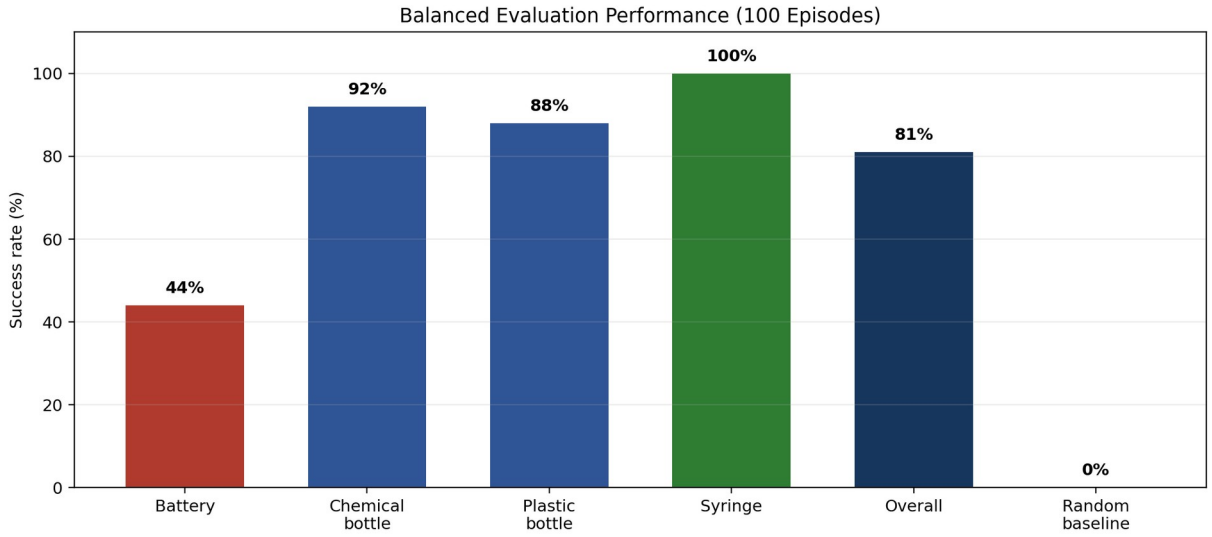


Figure 2. Per-task success, overall success, and random-action baseline.

Chemical-bottle and syringe sorting are the strongest outcomes, reaching 92% and 100% success respectively. Plastic-bottle sorting also performs strongly at 88%. Battery sorting remains the dominant failure mode at 44%, which explains most of the gap between the current 81% aggregate result and a uniformly high-performing policy. The final mean reward is 38.93 with a standard deviation of 19.87; mean episode length is 70.1 steps, and mean final bin distance is 0.158 m.

The per-task distribution is more informative than the overall score alone. It shows that the language-conditioned architecture can support several reliable behaviors, but robustness is not yet uniform across object geometry. The report therefore treats battery performance as an explicit limitation rather than hiding it within the aggregate metric.

7. Final Deliverables and Output Forms

The project is packaged as a reproducible GitHub submission. The final deliverables cover source code, trained artifacts, evaluation evidence, documentation, and a visual demonstration.

Table 5. Submission deliverables

Deliverable	Artifact	Purpose
Presentation-ready notebook	SafeSort_RL_FINAL_ PRESENTATION_READY.ipynb	Complete executable pipeline, environment, evaluation, and showcase.
Trained checkpoint	models/ safesort_genesis_amd_final_best.zip	Final PPO policy selected by the notebook.
Evaluation results	results/ final_evaluation.json	Machine-readable task metrics and evidence summary.
Environment evidence	results/ amd_environment.json and .txt	ROCm, GPU, Python, Genesis, and library versions.
Demonstration video	videos/ safesort_final_realistic_ sidepanel_showcase.mp4	Four-object workflow with synchronized side panel.
Reproducibility package	README.md, requirements.txt, setup_radeon.sh	Environment setup and step-by-step execution instructions.
Technical documentation	Technical paper and submission checklist	Architecture, methodology, results, limitations, and contribution record.

8. Strengths, Limitations, and Future Work

The project's main strength is that its language-conditioning claim is supported by a direct same-scene intervention rather than inferred only from aggregate accuracy. The evaluation is balanced and fixed-seed, the random baseline is explicit, and the weakest task is reported. The entire evidence pipeline is accessible from a single notebook and records the AMD runtime used to produce the results.

8.1 Current limitations

- Instructions are normalized symbolically; the system does not use a pretrained language model or open-ended semantic parser.
- Observations are compact state vectors rather than RGB or depth images.
- Grasp attachment and release are deterministic safety operations rather than learned end-to-end behaviors.
- The evidence is produced in simulation and has not yet been transferred to a physical robot.
- Battery sorting reaches 44%, making it the most important robustness gap in the current checkpoint.

8.2 Planned extensions

- Introduce camera-based perception and object detection while retaining state-based diagnostics for auditability.
- Apply visual, physical, and placement domain randomization to prepare for sim-to-real transfer.
- Learn grasp timing and release decisions, with safety constraints retained as fallback checks.
- Use a pretrained text encoder to support a broader set of natural-language instructions.
- Prioritize battery geometry, reward shaping, and curriculum coverage before expanding the task set.
- Validate the full pipeline on a physical Franka-class manipulator with controlled hazardous-object surrogates.

9. Team Member and Contribution Statement

This is a solo submission by Junie Mariam Varghese. The work includes the Genesis environment and Franka workcell design, language-conditioned task representation, PPO reward shaping and curriculum development, checkpoint evaluation, AMD Radeon and ROCm validation, instruction-swap grounding probe, physical weld-based showcase diagnostics, realistic object visualization, demonstration-video production, technical documentation, and reproducibility packaging.

10. Conclusion

SafeSort-RL demonstrates a complete physical-AI workflow for instruction-conditioned robotic sorting on AMD Radeon GPUs and ROCm. A single PPO policy controls four task intents in a Genesis Franka Panda environment, while an explicit safety layer provides reliable grasp attachment and release. The final system achieves 81% success over 100 balanced episodes and substantially outperforms a random-action baseline. Beyond the score, the project emphasizes transparent claim boundaries, per-task evaluation, direct instruction-grounding evidence, and reproducibility. These characteristics make the system a practical foundation for future camera-based perception, learned grasping, domain randomization, and real-robot validation.

Appendix A. Reproduction Summary

Evaluators should activate the provided AMD ROCm environment, verify the reported PyTorch HIP and Genesis versions, place the final checkpoint in the models directory, open the presentation-ready notebook, and execute the cells from top to bottom. Training is disabled by default to protect the verified checkpoint. Full evaluation, instruction-swap probing, AMD snapshots, benchmarks, and video regeneration can be enabled independently through the notebook configuration flags.

Expected primary outputs

A loaded final checkpoint, AMD environment evidence, optional 100-episode evaluation results, and the final four-object showcase video with the synchronized information panel.